
ProjectBuddy

AI-Enhanced Collaborative Project Management
Platform for Higher Education

Version: 1.0 | **Classification:** Public

Date: July 2026

Author: Safak Surmeli

Institution: International University of Sarajevo (IUS)

Contact: surmeliisafak@gmail.com

Abstract: This white paper presents ProjectBuddy, an open-source web-based platform designed to address the persistent challenges of team formation, project coordination, and peer assessment in higher education environments. Built on a modern Python/Flask technology stack with real-time communication capabilities, ProjectBuddy integrates AI-powered assistance (SSM-1.0), WebRTC voice collaboration, skill-based recommendation algorithms, and a gamified badge system to foster meaningful student collaboration. The platform has been developed and deployed at the International University of Sarajevo, serving as both a functional tool and a research contribution to the field of Computer-Supported Collaborative Learning (CSCL).

Table of Contents

1. Introduction

- 1.1 Problem Statement
- 1.2 Objectives and Scope
- 1.3 Target Audience

2. Literature Review and Related Work

- 2.1 Computer-Supported Collaborative Learning
- 2.2 Existing Platforms and Gap Analysis
- 2.3 AI in Educational Technology

3. System Architecture and Design

- 3.1 Architectural Overview
- 3.2 Technology Stack
- 3.3 Data Model
- 3.4 Application Factory Pattern
- 3.5 Security Architecture

4. Core Features and Implementation

- 4.1 User Management and Authentication
- 4.2 Project Lifecycle Management
- 4.3 Skill-Based Recommendation Engine
- 4.4 Community Learning Feed
- 4.5 Study Groups and Voice Chat
- 4.6 SSM-1.0 AI Assistant
- 4.7 Gamification and Badge System
- 4.8 Administrative Dashboard

5. Real-Time Communication Infrastructure

- 5.1 WebSocket Architecture
- 5.2 WebRTC Voice Implementation
- 5.3 Scalability with Redis Message Queue

6. Deployment and Operations

- 6.1 Production Configuration
- 6.2 Cloud Storage Abstraction
- 6.3 Background Task Scheduling

7. Evaluation and Discussion

- 7.1 Functional Assessment
- 7.2 Security Posture

7.3 Limitations and Known Constraints

8. Future Work

9. Conclusion

References

1. Introduction

The landscape of higher education has undergone a fundamental transformation in the 21st century. Collaborative project-based learning has emerged as a cornerstone pedagogical approach across computer science, engineering, and interdisciplinary programs worldwide. Research consistently demonstrates that students engaged in well-structured collaborative projects develop superior technical competencies, enhanced communication skills, and stronger professional networks compared to those in traditional lecture-based paradigms (Dillenbourg, 1999; Stahl et al., 2006).

However, the practical implementation of collaborative learning at scale presents persistent challenges that existing educational technology has failed to adequately address. Students struggle to find compatible teammates, project coordination often devolves into fragmented communication across multiple platforms, and meaningful peer assessment remains difficult to facilitate and verify.

ProjectBuddy was conceived and developed to address these systemic gaps. It is an open-source, web-based collaborative project management platform purpose-built for the higher education context. Unlike general-purpose tools such as Trello, Slack, or GitHub Projects, ProjectBuddy integrates the complete student collaboration lifecycle into a single, cohesive environment: from team discovery and formation, through real-time communication and file sharing, to peer feedback, skill endorsement, and AI-powered assistance.

1.1 Problem Statement

Through direct observation and student feedback at the International University of Sarajevo (IUS), we identified five critical pain points in collaborative academic projects:

- **Team Formation Friction:** Students rely on informal social networks to find project partners, creating information asymmetry where well-connected students form optimal teams while others are left with suboptimal matches or no team at all.
- **Platform Fragmentation:** A typical student project requires WhatsApp for messaging, Google Drive for documents, GitHub for code, and email for formal communication. This fragmentation leads to lost context, duplicated effort, and coordination overhead.
- **Absence of Structured Feedback:** Most project courses lack mechanisms for intra-team peer assessment. Free-riding is difficult to detect, and high-performing team members receive no recognition for their disproportionate contributions.
- **Skill Visibility Gap:** Students have no standardized way to showcase project-validated skills to future teammates or instructors. Academic transcripts capture grades but not competencies.
- **Limited Instructor Oversight:** Faculty supervising 20+ project teams simultaneously lack tooling to monitor team health, identify struggling groups, and intervene

proactively.

1.2 Objectives and Scope

ProjectBuddy aims to deliver a unified platform that addresses each of the identified pain points through the following design objectives:

- **O1 - Intelligent Team Formation:** Implement a skill-and-interest-based recommendation engine that surfaces compatible project opportunities to students, reducing the team formation search cost.
- **O2 - Unified Collaboration Environment:** Provide text chat, voice communication, file sharing, and project management within a single platform, eliminating the need for third-party tool fragmentation.
- **O3 - Structured Peer Assessment:** Enable post-project peer feedback with quantitative ratings (1-5) and qualitative comments, constrained by actual project membership to ensure assessment authenticity.
- **O4 - Skill Endorsement and Gamification:** Allow teammates to endorse specific skills after project completion, with an automated badge system that rewards sustained contribution.
- **O5 - AI-Powered Assistance:** Integrate an AI assistant (SSM-1.0) that provides contextual help for project planning, skill development, and platform navigation.
- **O6 - Administrative Tooling:** Equip instructors and administrators with dashboards for monitoring platform activity, managing reports, and overseeing AI chatbot interactions.

1.3 Target Audience

ProjectBuddy serves three distinct user roles, each with tailored functionality:

Role	Description	Key Capabilities
Student	Undergraduate and graduate students enrolled in project-based courses	Create/join projects, apply to teams, participate in voice chat, use AI assistant, give/receive feedback
Instructor	Faculty members supervising project courses (auto-assigned via faculty email domain)	All student capabilities plus instructor-level project ratings and enhanced visibility
Administrator	Platform administrators with full moderation authority	User management, report adjudication (warn/ban/dismiss), project removal, AI chat log monitoring, platform statistics

Table 1: ProjectBuddy User Roles and Capabilities

2. Literature Review and Related Work

2.1 Computer-Supported Collaborative Learning (CSCL)

Computer-Supported Collaborative Learning (CSCL) is a research paradigm that investigates how technology can facilitate group learning processes. Stahl, Koschmann, and Suthers (2006) define CSCL as the study of "how people can learn together with the help of computers," emphasizing that the unit of analysis is the group, not the individual. This distinction is fundamental to ProjectBuddy's design philosophy: every feature is evaluated against its impact on group dynamics, not merely individual productivity.

Dillenbourg (1999) established that effective collaborative learning requires four conditions: (1) a shared goal, (2) division of labor, (3) joint problem-solving, and (4) mutual knowledge construction. ProjectBuddy operationalizes each condition through its project lifecycle model: shared goals are defined at project creation, labor is divided through team role assignment, joint problem-solving is facilitated by real-time communication channels, and mutual knowledge construction is captured through skill endorsements and peer feedback.

More recently, Jeong and Hmelo-Silver (2016) proposed a framework for CSCL technology design that emphasizes seven affordances: (1) joint task performance, (2) communication, (3) resource sharing, (4) group awareness, (5) regulation, (6) engagement, and (7) assessment. ProjectBuddy's feature set maps directly to these affordances, as detailed in Section 4.

2.2 Existing Platforms and Gap Analysis

Several categories of software tools are currently used in academic collaborative settings. We conducted a comparative analysis to identify the specific gaps that motivated ProjectBuddy's development:

Platform Category	Examples	Strengths	Gaps for Academia
Project Management	Trello, Asana, Jira	Task tracking, kanban boards, deadline management	No team matching, no peer assessment, no academic role model, no voice chat
Communication	Slack, Discord, MS Teams	Real-time messaging, voice/video, channels	No project lifecycle, no skill tracking, no recommendation engine, no gamification
Version Control	GitHub, GitLab	Code collaboration, issue tracking, CI/CD	Code-centric (excludes non-CS disciplines), no peer feedback system, steep learning curve

Platform Category	Examples	Strengths	Gaps for Academia
LMS Platforms	Moodle, Canvas, Blackboard	Course management, grading, content delivery	Weak collaboration features, no real-time communication, no AI assistance, monolithic architecture
Social Academic	ResearchGate, Academia.edu	Research networking, paper sharing	Post-graduation focus, no project-level collaboration, no real-time features

Table 2: Comparative Analysis of Existing Platforms

The analysis reveals that no existing platform provides a unified solution that combines intelligent team formation, real-time multi-modal communication, structured peer assessment, skill endorsement, AI assistance, and administrative oversight within a single, purpose-built educational environment. ProjectBuddy occupies this unique intersection.

2.3 AI in Educational Technology

The integration of Large Language Models (LLMs) into educational platforms represents a rapidly evolving frontier. Recent work by Kasneci et al. (2023) surveys the opportunities and challenges of ChatGPT in education, noting that AI assistants can provide personalized scaffolding, reduce instructor workload for routine queries, and make domain expertise more accessible. However, they caution against uncritical deployment without safeguards for accuracy and academic integrity.

ProjectBuddy's AI component (SSM-1.0) is designed with these considerations in mind. Rather than replacing human interaction, SSM-1.0 serves as a supplementary resource for project planning advice, skill development recommendations, and platform guidance. Its conversation history is persisted and visible to administrators, maintaining transparency and accountability. The multi-provider architecture (Groq/LLaMA, Anthropic, with keyword-based fallback) ensures service continuity while controlling costs.

3. System Architecture and Design

3.1 Architectural Overview

ProjectBuddy follows a modular monolithic architecture organized around the Flask application factory pattern. While microservices architectures are prevalent in industry, we deliberately chose a monolithic approach for several reasons: (1) reduced operational complexity for a university-hosted deployment, (2) simpler debugging and development workflow, (3) atomic database transactions across features, and (4) the application's scale (thousands of users, not millions) does not require horizontal service decomposition.

The architecture is organized into four logical layers:

- **Presentation Layer:** Jinja2 server-rendered HTML templates with responsive CSS, vanilla JavaScript, and Socket.IO client for real-time updates.
- **Application Layer:** Flask blueprints (9 modules) implementing RESTful API endpoints and server-side rendering routes, with cross-cutting concerns (authentication, rate limiting, CSRF protection) handled by middleware and decorators.
- **Service Layer:** Business logic encapsulated in dedicated service modules (recommendation engine, badge system, deadline checker, file storage abstraction).
- **Data Layer:** SQLAlchemy 2.0 ORM with 25 mapped models, PostgreSQL in production, SQLite for development, managed through Alembic migrations.

3.2 Technology Stack

The technology stack was selected to balance developer productivity, ecosystem maturity, deployment simplicity, and long-term maintainability:

Layer	Technology	Version	Rationale
Web Framework	Flask	3.1	Lightweight, extensible, large ecosystem, well-suited for modular applications
ORM	SQLAlchemy	2.0	Industry-standard Python ORM with modern mapped_column syntax and type annotations
Database (Prod)	PostgreSQL	15+	ACID-compliant, robust JSON support, excellent concurrency handling
Database (Dev)	SQLite	3.x	Zero-configuration, file-based, ideal for local development iteration

Layer	Technology	Version	Rationale
Real-Time	Flask-SocketIO + eventlet	5.x	WebSocket abstraction with fallback, async worker integration
Message Queue	Redis	7.x	Cross-worker event broadcast, rate-limit storage, voice room state
Authentication	Flask-Login	0.6	Session-based auth with remember-me, mature and battle-tested
Migrations	Flask-Migrate (Alembic)	4.x	Versioned schema migrations, rollback capability
Rate Limiting	Flask-Limiter	3.x	Per-route rate limiting with Redis backend for multi-worker support
CSRF Protection	Flask-WTF	1.x	Token-based CSRF with configurable timeout
File Storage	boto3 (AWS S3)	1.x	Abstracted behind LocalStorage/S3Storage interface for portability
AI (Primary)	Groq API (LLaMA 3.3)	-	Free-tier LLM inference, OpenAI-compatible API format
AI (Secondary)	Anthropic API (Claude)	-	Fallback provider for high-quality responses
Voice (WebRTC)	Browser WebRTC + SocketIO signaling	-	Peer-to-peer audio with TURN relay support (Xirsys)
Deployment	Gunicorn + eventlet worker	-	Production WSGI server with async worker class for WebSocket support
Hosting	Render / Heroku	-	PaaS deployment via Procfile, managed PostgreSQL

Table 3: Complete Technology Stack

3.3 Data Model

The data model comprises 25 SQLAlchemy models organized into five functional domains. All models use the modern SQLAlchemy 2.0 `mapped_column` / `Mapped[type]` declarative syntax with full type annotations.

Domain	Models	Key Relationships
User Identity	User, UserInterest, UserSkill, UserCourse	User has many interests, skills, and courses; supports student/instructor/admin roles
Project Lifecycle	Project, ProjectTag, ProjectSkill, ProjectMember, Application, ProjectMessage, ProjectVote	Project owned by User; members join via Application; votes enable community ranking
Assessment	Feedback, Endorsement, Badge, UserBadge	Feedback links giver/receiver/project with rating constraint (1-5); endorsements require shared project completion
Communication	Chat, ChatMessage, StudyGroup, StudyGroupMember, StudyGroupMessage, SharedFile, ChatbotSession	Admin support chat (Chat/ChatMessage); group chat with file sharing; AI chatbot session persistence
Moderation	Report, AdminMessage, Notification, PasswordReset	Report targets user or project; admin can warn/ban/dismiss; notifications link to originating action

Table 4: Data Model Domain Organization

3.4 Application Factory Pattern

ProjectBuddy uses the Flask application factory pattern (`create_app()`) to ensure testability, configuration flexibility, and clean extension initialization. The factory performs the following operations in sequence:

- **Step 1:** Configuration loading from environment-specific classes (DevelopmentConfig, ProductionConfig, TestingConfig)
- **Step 2:** Werkzeug ProxyFix middleware installation for correct header handling behind reverse proxies (Render, Heroku)
- **Step 3:** Extension initialization: SQLAlchemy, Migrate, LoginManager, Limiter, CSRFProtect, SocketIO
- **Step 4:** Blueprint registration: 9 route modules plus SocketIO voice event handlers
- **Step 5:** Security middleware: CSP nonce generation, comprehensive security headers, cache-control policies
- **Step 6:** Startup tasks in OS thread: schema migration, badge seeding, admin sync, avatar backfill, mock data seeding
- **Step 7:** Background scheduler: hourly deadline checker running in a genuine OS thread (not eventlet greenlet)

3.5 Security Architecture

Security is implemented as a defense-in-depth strategy with multiple overlapping controls. Every HTTP response includes a comprehensive set of security headers:

Security Control	Implementation	Threat Mitigated
Content Security Policy	default-src 'self'; script-src 'self' 'unsafe-inline' cdn.jsdelivr.net; frame-ancestors 'none'	XSS, code injection, clickjacking
CSRF Protection	Flask-WTF token with 1-hour expiry; CSRF error returns flash + redirect (not raw 400)	Cross-Site Request Forgery
Rate Limiting	Global: 200/day, 50/hour; Login: 20/min, 100/hour; Register: 10/hour; Chatbot: 20/min, 200/day	Brute force, credential stuffing, API abuse
Password Security	PBKDF2-SHA256 hashing via Werkzeug; min 8 chars + digit + special character	Credential theft, rainbow table attacks
Session Security	HttpOnly, Secure, SameSite=Lax cookies; 8-hour session lifetime; __Host- prefix in production	Session hijacking, cookie theft
HSTS	max-age=63072000; includeSubDomains; preload (production only)	SSL stripping, downgrade attacks
OAuth CSRF	HMAC-signed timestamp state parameter (no session dependency); 10-minute window	OAuth state tampering, CSRF during GitHub login
Permissions Policy	camera=(), microphone=(self), geolocation=(), payment=()	Unauthorized browser API access
COOP/CORP	Cross-Origin-Opener-Policy: same-origin; Cross-Origin-Resource-Policy: same-origin	Cross-origin window attacks, resource embedding
File Upload Validation	Extension whitelist + Content-Type check + magic-byte sniffing + size limit	Malicious file upload, MIME confusion

Table 5: Security Controls Matrix

4. Core Features and Implementation

4.1 User Management and Authentication

ProjectBuddy supports two authentication pathways: traditional email/password registration and GitHub OAuth 2.0 single sign-on. The system implements automatic role assignment based on email domain: addresses matching a configurable faculty domain (e.g., @faculty.university.edu) receive the instructor role, while all other registrations default to the student role. Administrator accounts are provisioned through environment variables and synchronized at application startup.

The registration flow requires exactly five interest tags from a predefined taxonomy, which serve dual purposes: (1) populating the user's interest profile for the recommendation engine, and (2) providing an initial signal for team compatibility matching. Password policies enforce a minimum of 8 characters with at least one digit and one special character.

The GitHub OAuth implementation uses HMAC-signed timestamp-based state tokens rather than session-stored state, eliminating a class of bugs related to cross-site redirect cookie loss. The state token includes a 10-minute expiry window and is verified using constant-time comparison (`hmac.compare_digest`) to prevent timing attacks.

Password reset is implemented via a secure token-based flow: a 256-character random token is generated, stored in the database with a creation timestamp, and sent to the user's email via the Brevo transactional email API. Tokens expire after a configurable period (default: 1 hour). The reset endpoint does not reveal whether an email exists in the system, preventing enumeration attacks.

4.2 Project Lifecycle Management

Projects in ProjectBuddy follow a well-defined state machine with three states: **open** (accepting applications), **closed** (team formed, no new applications), and **completed** (deliverable finished, feedback enabled). Transitions are governed by business rules:

- **Open -> Closed:** Triggered automatically when the team reaches capacity, or manually by the project owner.
- **Open/Closed -> Completed:** Triggered manually by the project owner, or automatically by the background deadline checker when the deadline passes.
- **Completed:** Terminal state. Enables peer feedback and skill endorsement; disables project modification and new applications.

Each project creation requires exactly five topic tags from a predefined taxonomy, a team size specification, and an optional deadline. The system enforces a hard limit of three concurrent active project memberships per student, preventing overcommitment. The application workflow

includes applicant messaging, owner review (accept/reject), and automatic notifications.

Community engagement is encouraged through a voting system: users can upvote or downvote projects, with a unique constraint ensuring one vote per user per project. The resulting vote score provides a community-curated quality signal for project discovery.

4.3 Skill-Based Recommendation Engine

The recommendation engine (`services/recommendation_service.py`) implements a content-based filtering algorithm that matches student interest profiles against project topic tags. The algorithm operates in three phases:

Phase 1 - Interest Expansion: Each user's selected interest tags are expanded through a curated mapping dictionary (`INTEREST_TO_TOPIC_MAP`) that associates narrow interests with broader topic categories. For example, "machine learning" expands to {"ai / ml", "data", "research"}, and "react" expands to {"frontend"}. This expansion increases recall by bridging vocabulary gaps between how students describe their interests and how project owners tag their projects.

Phase 2 - Candidate Filtering: The algorithm filters out projects where the user is already the owner, an active member, or has a pending application. Only projects with "open" status are considered.

Phase 3 - Scoring and Ranking: For each remaining candidate project, the algorithm computes the intersection cardinality between the user's expanded interest set and the project's normalized tag set. Projects are ranked by (`match_count DESC`, `created_at DESC`), prioritizing relevance while using recency as a tiebreaker.

4.4 Community Learning Feed

The community module provides a social learning feed modeled after educational discourse platforms. Students can publish posts with optional media attachments (images: JPG, PNG, WebP, GIF; videos: MP4, WebM, MOV) up to 50 MB. The upload pipeline implements four-layer validation: extension whitelist, Content-Type header verification, magic-byte signature sniffing (for images), and file size enforcement.

Posts support a comment thread with @mention notifications. When a user comments on another user's post, the post author receives an in-app notification with a preview snippet of the comment. A like/unlike toggle provides lightweight engagement signaling. Both posts and comments can be deleted by their authors or by administrators, supporting content moderation.

4.5 Study Groups and Voice Chat

Study groups provide persistent, topic-organized collaboration spaces with three core capabilities: text-based group messaging (polling-based with 3-second intervals, last 60 messages loaded on entry), file sharing (25 MB limit, 40+ supported formats), and peer-to-peer WebRTC voice chat.

The voice chat system is detailed in Section 5. The file sharing subsystem uses the same storage abstraction as the community feed, supporting both local disk storage (development) and S3-compatible cloud storage (production) through the `services/file_storage.py` module. Files are stored with UUID-based filenames to prevent collisions and path traversal, while preserving the original filename for display and download purposes.

4.6 SSM-1.0 AI Assistant

SSM-1.0 is ProjectBuddy's integrated AI assistant, available both as a web-based chatbot within the platform and as a standalone CLI development tool. The web component operates through a multi-provider architecture with automatic failover:

Provider	Model	Tier	Characteristics
Groq (Primary)	LLaMA 3.3 70B Versatile	Free	Fast inference, OpenAI-compatible API, 14,400 req/day free tier
Anthropic (Fallback)	Claude 3 Haiku	Paid	High-quality responses, Anthropic Messages API format
Built-in (Offline)	Keyword matching	Free	Category-based response templates, zero API dependency

Table 6: SSM-1.0 Provider Architecture

Each user's conversation history is persisted in the database (last 20 messages retained per user) and included in subsequent API calls to maintain conversational context. The chatbot is rate-limited to 20 requests per minute and 200 per day per user. Input is validated for length (max 1000 characters) and emptiness before processing.

The standalone CLI agent (`ssm_agent.py`) implements a ReAct-style (Reasoning + Acting) agent loop over a local Ollama model. It includes a tool registry that supports file I/O, shell command execution, and datetime queries. The agent parses tool calls from model output as JSON, executes them, and feeds results back into the conversation for iterative reasoning, with a maximum of 10 tool-call rounds per user message.

4.7 Gamification and Badge System

ProjectBuddy implements a badge-based gamification system (`services/badge_service.py`) designed to incentivize sustained engagement and recognize achievement milestones:

Badge	Condition	Design Rationale
First Step	Complete 1 project	Lower the activation barrier; reward initial participation
Veteran	Complete 5 projects	Recognize sustained commitment across multiple teams
Expert	Receive 10 skill endorsements	Validate peer-recognized expertise; require social proof

Table 7: Badge System Design

Badges are automatically evaluated after two trigger events: project completion (for First Step and Veteran) and skill endorsement (for Expert). The evaluation is idempotent: if a badge has already been awarded, the check short-circuits without database modification. Endorsements are constrained to users who have completed at least one project together, ensuring that endorsements reflect genuine collaborative experience rather than social reciprocity.

4.8 Administrative Dashboard

The admin module provides a dedicated interface for platform governance. Key capabilities include:

- **Report Adjudication:** Three-action workflow for user reports: warn (sends a warning AdminMessage to the target user), ban (disables login), or dismiss (closes the report as invalid). Reports follow a strict pending -> resolved/warned/dismissed state machine.
- **Platform Statistics:** Real-time aggregate metrics including total users, projects, applications, reports, and badges. Accessible via both the dashboard UI and a JSON API endpoint.
- **AI Chat Monitoring:** Full read access to all users' chatbot conversation histories, organized by user. This enables quality assurance of AI responses and detection of potential misuse patterns.
- **Direct Messaging:** Admin-to-user communication channel for warnings, guidance, and follow-up on reported issues.
- **Project Removal:** Ability to remove fraudulent or misleading project listings, with cascading deletion of associated tags, members, and applications.

5. Real-Time Communication Infrastructure

5.1 WebSocket Architecture

ProjectBuddy's real-time layer is built on Flask-SocketIO with the eventlet async mode. The eventlet worker class patches Python's standard library to provide cooperative multitasking within a single OS process, allowing thousands of concurrent WebSocket connections without the thread-per-connection overhead of traditional WSGI servers.

A critical architectural decision is the use of genuine OS threads (via `eventlet.patcher.original('threading').Thread`) for background tasks such as the deadline scheduler and database startup routines. This is necessary because eventlet's hub-based scheduling raises "do not call blocking functions from the mainloop" when `hub.switch()` is invoked from the hub's own greenlet. By using unpatched OS threads, these tasks execute completely outside eventlet's cooperative scheduling, avoiding deadlocks while maintaining access to the Flask application context.

5.2 WebRTC Voice Implementation

The voice chat system implements a full-mesh peer-to-peer WebRTC topology with server-side signaling. The implementation follows a five-phase connection lifecycle:

- **Phase 1 - Room Join:** Client emits `join_voice` with `group_id`. Server joins the SocketIO room, retrieves the existing participant list, and sends it to the new joiner via `voice_existing_participants`.
- **Phase 2 - Offer Exchange:** Existing participants receive `voice_user_joined` and initiate WebRTC offers. SDP offers are relayed through the server to the target peer identified by socket ID.
- **Phase 3 - Answer Exchange:** The new joiner generates SDP answers and relays them back through the server. At this point, both peers have exchanged session descriptions.
- **Phase 4 - ICE Candidate Exchange:** ICE (Interactive Connectivity Establishment) candidates are relayed between peers to negotiate the optimal network path. With TURN configured, this ensures connectivity even behind symmetric NATs and restrictive firewalls.
- **Phase 5 - Teardown:** On `leave_voice` or `disconnect`, the participant is removed from the room state and all remaining peers are notified via `voice_user_left`.

Voice room state is stored in Redis when available (multi-worker production deployments) or in an in-process dictionary (single-worker development). Redis entries use a 24-hour TTL to

prevent stale state accumulation.

ICE server configuration supports three tiers: (1) Xirsys API for fresh, time-limited TURN credentials on each room load; (2) static TURN configuration (Metered, Twilio, Coturn) via environment variables; (3) Google STUN-only fallback for same-LAN development. The system logs a warning if no TURN relay is configured, as STUN alone is insufficient for production use behind NAT/firewalls.

5.3 Scalability with Redis Message Queue

In production, Gunicorn runs multiple eventlet worker processes to utilize multi-core hardware. This creates a challenge: SocketIO events emitted in one worker process must reach clients connected to other workers. ProjectBuddy solves this by configuring Flask-SocketIO with a Redis message queue (`message_queue=redis_url`), enabling cross-worker event broadcast.

Redis also serves as the backend for Flask-Limiter's rate limiting storage, ensuring that rate limits are enforced consistently across all worker processes. In development (single worker), the system gracefully degrades to in-process storage for both SocketIO events and rate limits.

6. Deployment and Operations

6.1 Production Configuration

ProjectBuddy uses a three-tier configuration hierarchy: base Config, DevelopmentConfig, and ProductionConfig. The production configuration enforces several critical settings that differ from development:

- **Secure Cookies:** `SESSION_COOKIE_SECURE=True`, `SESSION_COOKIE_HTTPONLY=True`, `SameSite=Lax`
- **Mock Data Disabled:** `SEED MOCK DATA=False` prevents test data from being injected
- **HSTS Enabled:** 2-year max-age with includeSubDomains and preload directives
- **CORS Restricted:** SocketIO CORS origins are limited to the deployment URL (from `RENDER_EXTERNAL_URL` or `CORS_ORIGINS` env var)
- **Debug Disabled:** Stack traces and interactive debugger are not exposed

The deployment pipeline uses a Procfile-based configuration compatible with Render and Heroku. The WSGI entrypoint (`wsgi.py`) initializes the application with ProductionConfig and runs under Gunicorn with the eventlet worker class.

6.2 Cloud Storage Abstraction

File storage is abstracted behind a pluggable interface (`services/file_storage.py`) with two implementations:

LocalStorage: Saves files to `static/uploads/<category>/` on the local filesystem. Returns relative URLs suitable for direct serving by the web server. Used in development and single-server deployments.

S3Storage: Uploads files to an AWS S3 bucket (or S3-compatible service via custom endpoint URL). Supports CloudFront CDN URL rewriting for optimized delivery. Generates pre-signed download URLs with configurable expiry for access-controlled file retrieval. Content-Type is automatically detected via MIME type guessing.

The storage backend is selected at module import time based on the presence of the `AWS_S3_BUCKET` environment variable, with automatic fallback to local storage if S3 initialization fails. This design ensures zero-configuration development while supporting production cloud storage without code changes.

6.3 Background Task Scheduling

ProjectBuddy runs a single background task: the deadline checker, which executes hourly to identify overdue projects and automatically transition them to the "completed" state. The scheduler is implemented as a genuine OS thread (not an eventlet greenlet) to avoid the "blocking from mainloop" issue described in Section 5.1.

In Flask's debug mode with the Werkzeug reloader, the scheduler is conditionally started only in the child process (detected via the `WERKZEUG_RUN_MAIN` environment variable) to prevent duplicate scheduler instances from competing over database access.

7. Evaluation and Discussion

7.1 Functional Assessment

We evaluate ProjectBuddy against the CSCL affordances framework proposed by Jeong and Hmelo-Silver (2016):

CSCL Affordance	ProjectBuddy Implementation	Coverage
Joint Task Performance	Project lifecycle management with shared membership, role assignment, and state transitions	Full
Communication	Text messaging (study groups), WebRTC voice chat, admin messaging, AI chatbot	Full
Resource Sharing	File upload/download in study groups (40+ formats, 25 MB limit), community media posts	Full
Group Awareness	Live voice presence indicators, member lists, application status tracking, notification system	Partial
Regulation	Deadline enforcement (automatic), project membership limits (max 3), rate limiting	Partial
Engagement	Badge gamification, skill endorsements, community likes/comments, project voting	Full
Assessment	Peer feedback (1-5 rating + comment), skill endorsements, instructor ratings	Full

Table 8: CSCL Affordance Coverage Assessment

Group Awareness is rated "Partial" because while the platform provides real-time voice presence and member status, it does not yet implement task progress dashboards or contribution analytics that would provide deeper insight into individual team member activity. Similarly, Regulation is "Partial" because while deadlines and membership limits are enforced, the platform does not yet support instructor-defined milestones or automated progress gates.

7.2 Security Posture

ProjectBuddy implements defenses against the OWASP Top 10 (2021) web application security risks. The comprehensive security header suite (CSP, HSTS, COOP, CORP, Permissions-Policy, X-Frame-Options, X-Content-Type-Options) addresses injection, misconfiguration, and cross-origin attack vectors. The four-layer file upload validation (extension + Content-Type + magic-byte + size) provides robust defense against malicious file

upload attacks.

Authentication security includes PBKDF2-SHA256 password hashing, rate-limited login endpoints, HMAC-signed OAuth state parameters, and secure password reset tokens with configurable expiry. Session cookies use HttpOnly, Secure, and SameSite attributes with an 8-hour lifetime. The CSRF protection covers all state-changing endpoints with token-based validation.

7.3 Limitations and Known Constraints

We acknowledge several limitations of the current implementation:

- **Full-Mesh Voice Topology:** The peer-to-peer WebRTC mesh topology has $O(n^2)$ connection complexity, making it unsuitable for rooms with more than 6-8 participants. A selective forwarding unit (SFU) architecture would be needed for larger rooms.
- **Polling-Based Group Chat:** Study group messages use HTTP polling (3-second intervals) rather than WebSocket push. While sufficient for the current scale, this creates unnecessary server load and latency compared to SocketIO-based message delivery.
- **Single-Server Recommendation Engine:** The recommendation algorithm queries all open projects on each request ($O(n)$ projects). For very large deployments, this would need indexing or caching.
- **No Automated Testing:** The current codebase lacks a comprehensive test suite. Unit tests for business logic and integration tests for API endpoints are high-priority technical debt.
- **Limited Accessibility:** While the UI is responsive, it has not undergone formal WCAG 2.1 accessibility auditing.
- **AI Response Quality:** The free-tier Groq/LLaMA models may produce lower-quality responses compared to commercial alternatives. The keyword-based fallback provides minimal utility.

8. Future Work

Based on the limitations identified and user feedback collected during the initial deployment at IUS, we propose the following roadmap for future development:

- **SFU-Based Voice Architecture:** Migrate from full-mesh WebRTC to a Selective Forwarding Unit (e.g., mediasoup or Janus) to support voice rooms with 20+ concurrent participants while reducing client-side bandwidth requirements.
- **SocketIO-Based Group Messaging:** Replace the polling-based study group chat with real-time SocketIO event delivery, reducing latency and server load while enabling typing indicators and read receipts.
- **Machine Learning Recommendations:** Enhance the recommendation engine with collaborative filtering based on historical team formation patterns, project completion rates, and feedback scores. Explore embedding-based similarity using project descriptions and user profiles.
- **Contribution Analytics Dashboard:** Implement per-project activity dashboards showing individual member contributions (messages sent, files shared, time in voice chat) to support instructor oversight and fair assessment.
- **Comprehensive Test Suite:** Develop unit tests (pytest) for all service modules, integration tests for API endpoints, and end-to-end tests (Selenium/Playwright) for critical user flows. Target 80%+ code coverage.
- **WCAG 2.1 AA Compliance:** Conduct a formal accessibility audit and remediate identified issues including keyboard navigation, screen reader compatibility, and color contrast ratios.
- **Mobile Application:** Develop a React Native or Flutter companion application to provide native push notifications, offline access to project information, and optimized voice chat on mobile networks.
- **LTI Integration:** Implement Learning Tools Interoperability (LTI 1.3) to enable seamless embedding within institutional LMS platforms (Moodle, Canvas) and automated grade passback.
- **Multi-Language Support:** Internationalize the platform with i18n support, starting with English, Turkish, and Bosnian as the primary languages of the IUS student population.

9. Conclusion

This white paper has presented ProjectBuddy, an open-source web-based platform that addresses the fundamental challenges of collaborative project-based learning in higher education. By integrating intelligent team formation, real-time multi-modal communication, structured peer assessment, skill endorsement, AI-powered assistance, and administrative tooling into a single cohesive environment, ProjectBuddy represents a significant contribution to the Computer-Supported Collaborative Learning (CSCL) technology landscape.

The platform's technical architecture demonstrates that modern web technologies — Flask, SQLAlchemy 2.0, WebRTC, Redis, and large language models — can be composed into a robust, secure, and maintainable system suitable for university-scale deployment. The defense-in-depth security approach, with its comprehensive header suite, rate limiting, CSRF protection, and multi-layer file validation, establishes a security baseline appropriate for handling student data in an educational context.

The skill-based recommendation engine, while algorithmically straightforward, effectively bridges the team formation gap by matching students with compatible projects based on their declared interests. The gamification system provides tangible incentives for sustained participation, and the AI assistant (SSM-1.0) offers contextual support without replacing the human collaboration that is the platform's core value proposition.

ProjectBuddy is deployed and actively used at the International University of Sarajevo, where it continues to evolve based on student and faculty feedback. The codebase is maintained as an open-source project, inviting contributions from the educational technology community. We believe that purpose-built collaboration platforms, informed by CSCL research and implemented with production-grade engineering practices, can meaningfully improve the collaborative learning experience for students worldwide.

References

-
- [1] Dillenbourg, P. (1999). *Collaborative Learning: Cognitive and Computational Approaches*. Elsevier Science. ISBN: 978-0-08-043073-7.
 - [2] Jeong, H., & Hmelo-Silver, C. E. (2016). Seven affordances of computer-supported collaborative learning: How to support collaborative learning? How can technologies help? *Educational Psychologist*, 51(2), 247-265. doi:10.1080/00461520.2016.1158654.
 - [3] Kasneci, E., Sessler, K., Kuchemann, S., et al. (2023). ChatGPT for good? On opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103, 102274. doi:10.1016/j.lindif.2023.102274.
 - [4] OWASP Foundation. (2021). *OWASP Top 10 - 2021*. Available at: <https://owasp.org/www-project-top-ten/>.
 - [5] Stahl, G., Koschmann, T., & Suthers, D. (2006). Computer-supported collaborative learning: An historical perspective. In R. K. Sawyer (Ed.), *Cambridge Handbook of the Learning Sciences* (pp. 409-426). Cambridge University Press.
 - [6] Yew, E. H. J., & Goh, K. (2016). Problem-based learning: An overview of its process and impact on learning. *Health Professions Education*, 2(2), 75-79. doi:10.1016/j.hpe.2016.01.004.
 - [7] Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media. ISBN: 978-1-491-99173-2.
 - [8] Bayer, M. (2023). SQLAlchemy 2.0 Documentation: Mapped Column Declarations. Available at: <https://docs.sqlalchemy.org/en/20/>.
 - [9] Mozilla Developer Network. (2024). WebRTC API Reference. Available at: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API.
 - [10] Flask-SocketIO Documentation. (2024). Flask-SocketIO: Socket.IO integration for Flask applications. Available at: <https://flask-socketio.readthedocs.io/>.